

Interval-Weighted Affine Orbit Profiles and Sparse Attachment Replay

Certified combinatorial kernels for unknot-recognition hierarchies

Research contribution for the ProveIt project

9 October 2026

Abstract

We study a precise compressed component-query problem motivated by the attachment bookkeeping in unknot-recognition hierarchies. An explicitly represented finite base graph carries fibres of binary size W and full-fibre identifications $x \mapsto \pm x + a \pmod{W}$. Each of K interval records contributes a signed integer vector at its points. We prove that a connected base block has at most $2K + 3$ distinct orbit-weight vectors, with binary multiplicities, and that this bound is attained for every $K \geq 1$, even by nonnegative scalar interval weights. The proof uses a reflection quotient of a discrete cycle, not an enumeration of its residues.

For b base blocks and h additional pointwise attachments, including additive edge payloads, the complete weighted histogram has support at most $2K + 3b + h$. Exact attachment replay uses a graph on at most $2h$ named bulk orbits; it retains sheet coordinates rather than replacing attaching data by abstract component types. In a fixed-base insertion-only epoch, at most $b(1 + \lfloor \log_2 W \rfloor)$ effective affine-subgroup changes require profile reconstruction, irrespective of the number of inserted maps. A guard verifies supplied full-fibre decompositions from ordinary interval pairings. A standard-library implementation, a separately implemented arithmetic certificate checker, exhaustive and randomized expansion audits, and reproducible kernel measurements accompany the proofs.

These results specialize and refine, rather than replace, the general weighted orbit algorithms of Agol–Hass–Thurston and ProveIt’s existing dihedral-cover classifier. They establish polynomial-bit-cost component kernels on the stated input class. They do not establish a general quasi-polynomial unknot recognizer, a geometric producer for this class, or a bound on the number of hierarchy states.

Contents

1	Research objective and relationship to the repository	4
1.1	The bottleneck addressed	4
1.2	What is new here, and what is reused	4
1.3	Current external complexity context	5
2	The exact input and output contract	5
2.1	Required queries	6
2.2	Size parameters and representation limits	6
3	Gauge reduction and the reusable bulk index	6
3.1	A spanning forest fixes common coordinates	6
3.2	Translation subgroup and reflection pairing	7
3.3	Canonical keys and local weight transport	7
4	The sharp interval-profile theorem	8
4.1	Residue counts are step functions	8
4.2	Reflection quotient and a stronger support bound	8
4.3	A family attaining the bound	10
4.4	Representatives without component enumeration	11
5	Sparse pointwise attachment replay	11
5.1	The online invariant	12
5.2	Why coordinates cannot be replaced by profile names	12
6	Insertion-only epochs and logarithmically many rebuilds	13
6.1	A precise amortized accounting statement	14
6.2	Stale keys, cancellation, and epochs that really end	14
7	A checked front end for ordinary interval pairings	15
8	Bit complexity and certificate soundness	15
8.1	No numeric-size iteration	15
8.2	Certificate contents	16
8.3	Trusted boundary and limits of replay	17
9	A conditional surface interpretation and disc completion	17
10	Validation and measured kernel behavior	18
10.1	Checks actually executed	18
10.2	Paired experiment design	19
10.3	Large-bit complete certificate runs	19
11	Repository integration and the quasi-polynomial target	20
11.1	Proposed integration boundary	20
11.2	What the kernel contributes to a complexity proof	20
11.3	Three nonimplications that must survive integration	21

12 Further research questions and proposed next steps	21
12.1 A checked geometric block producer	21
12.2 Compressed residual intervals rather than point defects	21
12.3 Variable-width fibres and arithmetic interfaces	21
12.4 Selective rebasing and a dependency graph	21
12.5 Deletions, cuts, and epoch potentials	22
12.6 Peripheral data and orientation characters	22
12.7 Beyond signed translations	22
12.8 Certificates for incremental histories	22
12.9 Exact bridge to disc-completion representative bases	22
12.10 Native benchmarks and falsifiable performance criteria	22
12.11 The remaining global theorem	23
13 Conclusion	23
A Source audit and comparison boundary	23
B Reproduction and artifact map	24

1 Research objective and relationship to the repository

1.1 The bottleneck addressed

A compressed description can name exponentially many parallel pieces while using only a few binary integers. Such a description is useful only if subsequent operations preserve its advantage. Replacing a component classification by an explicit list of components, or repeating a complete orbit computation after every small attachment, can lose the compression at precisely the interface where a hierarchy needs it.

The present contribution isolates three operations: compute the distribution of additive component data, attach specified points with exact provenance, and update the ambient full-fibre equivalence relation without rebuilding on redundant generators. The mathematical object is a finite equivalence system; no three-manifold is implicitly manufactured from that object. A future geometric caller must establish the correspondence between its pieces and this system.

The motivating repository is `Topology/UnknotRecognition` in ProveIt. Its maintained surface-cover work already gives a binary signed-affine monodromy classifier and exact ordered-point comparisons over a fixed base [5, 6]. In particular, that work explicitly distinguishes an isomorphism type from the actual placement of attachment points. Our weighted histograms must not erase this distinction. The maintained recognizer also explicitly declines to claim an implemented general quasi-polynomial bound [4].

1.2 What is new here, and what is reused

The algebraic description of the translation subgroup by a gcd, the reflection pairing of residue classes, and the interpretation of graph-cover components as monodromy orbits are reused background. Covering-space classification is classical [2, Section 1.3]; the relevant signed-affine specialization is already present in the repository [6].

Agol–Hass–Thurston give polynomial orbit counting for interval pairings and, in Theorem 16 of their paper, a weighted version with compressed output [1]. Their Corollary 17 applies the method to normal-surface component information. It would therefore be incorrect to describe the result below as the first polynomial weighted component algorithm, or to compare it only with sheet expansion and infer superiority to Agol–Hass–Thurston. Our scope is narrower: globally compatible signed-affine fibres, a sharp support bound for the weight distribution, an exact sparse-attachment interface, and a fixed-base amortization theorem.

The contribution consists of the following statements, all proved below. The first is the central combinatorial result.

1. The support bound $2K + 3$ is valid and sharp for a connected affine block, including signed vector weights and both reflection fixed-point cases (Theorems 4.3 and 4.5).
2. Sparse pointwise attachments give a finite quotient computation and increase the support bound by at most their number (Theorem 5.1).
3. A monotone epoch has only logarithmically many effective subgroup updates per base block, although every supplied generator still must be read (Theorem 6.1).
4. A directly checkable interval-pairing guard and an independent arithmetic replay certificate implement the restricted contract (Theorems 7.1 and 8.1).

The histogram bound and attachment quotient concern actual orbit weights, not merely an invariant of a knot. Their eventual use inside recognition is nevertheless conditional on a checked geometric producer. Section 11 identifies that missing interface rather than assuming it.

1.3 Current external complexity context

Lackenby’s July 2026 preprint gives a hierarchy-based incompressibility algorithm and its unknot-recognition application [3]. Section 9 separates the number of algorithmic steps from the cost of executing them, and records that some steps are not specified finely enough there to estimate their running times. That distinction is precisely why a small, executable, bit-costed interface is worth developing. It does not justify assigning our restricted bound to the entire hierarchy algorithm.

Our source review was pinned to the repository revision recorded in Appendix A. The incoming directory was inspected at the catalogue and archive-metadata level. Several current disc-completion and signed-continuation archives were visible, but their complete contents were not obtained in this session. Accordingly, we do not assert a comprehensive novelty comparison against those archives. The direct comparison supporting this manuscript is with the fetched maintained cover interface and its full theory section. The new package leaves both the maintained implementation and incoming archives unchanged.

2 The exact input and output contract

Definition 2.1 (Uniform signed-affine fibre system). Let $G = (V, E)$ be an explicitly represented finite multigraph, with $v = |V| \geq 1$ and $e = |E|$. Loops and repeated edge records are allowed. Choose a positive integer W , encoded in binary. Each vertex u carries a fibre

$$X_u = \{u\} \times \mathbb{Z}/W\mathbb{Z}.$$

Each oriented edge record $a : u \rightarrow z$ contains a sign $\varepsilon_a \in \{1, -1\}$ and a shift $t_a \in \mathbb{Z}/W\mathbb{Z}$. It imposes the undirected identifications

$$(u, x) \sim (z, \varepsilon_a x + t_a) \quad (x \in \mathbb{Z}/W\mathbb{Z}). \quad (1)$$

The equivalence classes generated by all these identifications are the *bulk orbits*.

An edge denotes a bijection on the entire fibre, not a partial interval map. The inverse has sign ε_a and shift $-\varepsilon_a t_a$. The word “oriented” only identifies which formula is supplied; it does not mean that reachability is directed. Nor does an affine sign automatically represent an orientation character of a surface.

Definition 2.2 (Interval weights and histogram). Fix a dimension $D \geq 1$. A weight record consists of a vertex u , an ordinary half-open integer interval $[l, r) \subseteq [0, W)$, and a vector $z \in \mathbb{Z}^D$. It contributes z at every point (u, x) with $l \leq x < r$. There are K records, which may overlap. Contributions add, with no positivity assumption. For a bulk orbit O , write $w(O)$ for the sum of all point weights in O . The histogram is the finite map

$$H(a) = \#\{O : w(O) = a\}, \quad a \in \mathbb{Z}^D,$$

with zero-multiplicity entries omitted. Its support is the set of distinct weight vectors, not the set of orbits.

Weights equal to zero are permitted and are not used as an “absent component” marker. A system with no nonzero point weights still has a histogram entry at the zero vector, with the correct number of components.

Definition 2.3 (Sparse attachments). A pointwise attachment is a record $((u, x), (z, y), p)$ with $p \in \mathbb{Z}^D$. It joins the named points and contributes the vector p once to the resulting connected component. There are h such records. Repeated attachments and self-attachments are permitted, and each record has its own payload. The *final histogram* sums bulk point weights and all attachment payloads in each final component.

The payload convention is a definition of additive bookkeeping. A geometric application must prove that its operation has precisely that additive effect. For example, a certified gluing of two disjoint boundary arcs changes Euler characteristic by -1 ; identifying arbitrary points of an unspecified geometric object is not automatically such a gluing.

2.1 Required queries

The implementation returns the complete compressed histogram, the component count, the exact component key of a supplied point, its total component weight, and connectivity of supplied pairs after attachments. It can also return one actual fibre point in a component with any requested histogram value, or report that none exists. It does not output all components individually.

The distinction matters: W isolated points with identical weights have a one-entry histogram, but listing all W witnesses necessarily costs $\Omega(W)$. Our output contract never requires such a list. Representatives are only generated on demand.

2.2 Size parameters and representation limits

We use v, e, K, D, h as explicit structural parameters and write b for the number of connected components of the base graph. The sheet count W , interval endpoints, shifts, and weight coordinates are binary integers. A bit parameter B includes their maximum bit length and $\lceil \log_2(v+e+K+h+2) \rceil$. The dimension is charged explicitly.

An important implementation detail is that the Python schema declares `vertices` as an integer but allocates one record per base vertex. Thus our bounds are polynomial in v , not in $\log v$ alone for a hypothetically binary-compressed collection of unlisted isolated base vertices. This is the ordinary explicit-base-graph regime relevant to the proposed interface. The compression theorem concerns W and the lifted orbit count. The command line additionally enforces a configurable explicit-vertex cap.

3 Gauge reduction and the reusable bulk index

3.1 A spanning forest fixes common coordinates

Choose a rooted spanning tree in each connected base block. Transport the root fibre along tree paths. At vertex u this gives

$$\phi_u(r) = s_u r + a_u \pmod{W}, \quad s_u \in \{1, -1\}, \quad (2)$$

with $s_\rho = 1, a_\rho = 0$ at a root ρ . Traversing an edge of sign ε and shift t updates (s, a) to $(\varepsilon s, \varepsilon a + t)$. The entire forest and its transports take $O(v + e)$ modular operations.

For any edge $a : u \rightarrow z$, its residual root-fibre map is

$$\phi_z^{-1} \circ a \circ \phi_u(r) = s_z \varepsilon_a s_u r + s_z (\varepsilon_a a_u + t_a - a_z) \pmod{W}. \quad (3)$$

Tree edges have identity residual maps, which may be retained harmlessly.

Lemma 3.1 (Orbit reduction). *Two lifted points in one connected base block are equivalent if and only if their root coordinates lie in the same orbit of the group generated by all residual maps (3). In particular, no operation on the lifted graph is needed to recover bulk connectivity.*

Proof. The tree identifications join $(u, \phi_u(r))$ to (ρ, r) for every u, r . Conjugating any original edge identification by these tree paths gives precisely its residual identification. Conversely, each residual identification is a concatenation of original edge and inverse-tree identifications. The generated equivalence relations therefore agree under root-coordinate transport. $\square$

3.2 Translation subgroup and reflection pairing

Consider one base block. Write its residual maps as $r \mapsto \varepsilon_i r + t_i$. If all signs are positive, define

$$d = \gcd(W, t_1, \dots, t_q). \quad (4)$$

If a negative sign occurs, choose one such shift c_0 and set

$$d = \gcd(W, \{t_i : \varepsilon_i = 1\}, \{t_i - c_0 : \varepsilon_i = -1\}), \quad c = c_0 \bmod d. \quad (5)$$

The case with no residual generators has $d = W$.

Lemma 3.2 (Signed-affine normal form; reused algebra). *The translation subgroup is generated by $r \mapsto r + d$. In the positive case, bulk orbits are indexed by residues modulo d . In the signed case, they are indexed by the orbits of*

$$R(r) = c - r \pmod{d}. \quad (6)$$

If f is the number of solutions of $2r = c \pmod{d}$, the bulk orbit count is d in the positive case and $(d + f)/2$ in the signed case. Here $f \leq 2$.

Proof. Positive generators are translations. A product of two negative generators is a translation by the difference of their shifts. Conjugation by a reflection negates a translation. Thus the listed increments generate every translation, and Bézout's identity gives the subgroup $d\mathbb{Z}/W\mathbb{Z}$ in both directions. Fixing one reflection then expresses every negative element as that reflection composed with a translation.

Translation orbits are residue classes modulo d . A reflection maps class r to class $c - r$. These classes are either paired or fixed. A fixed residue solves the displayed congruence, whose solution count is at most $\gcd(2, d) \leq 2$. Pairing the remaining $d - f$ residues gives the count. $\square$

This proof is included to make the package self-contained, not as a new claim over the repository's existing cover classifier. It also explains why preserving one global coordinate system is an essential hypothesis. Independently normalizing unrelated pieces without retaining their transports would not make their attachment coordinates comparable.

3.3 Canonical keys and local weight transport

For a point (u, x) with base root ρ , its root coordinate is

$$r = s_u(x - a_u) \pmod{W}.$$

The exact bulk key is $(\rho, r \bmod d)$ in the positive case and

$$(\rho, \min(r \bmod d, (c - r) \bmod d)) \quad (7)$$

in the signed case. Equality of these keys is connectivity in the supplied system, not an isomorphism between arbitrary ambient pieces.

A weight interval $[l, r)$ at u pulls back to a cyclic root interval of the same length $\ell = r - l$. Its initial root coordinate is

$$a = \begin{cases} l - a_u \pmod{W}, & s_u = 1, \\ a_u - r + 1 \pmod{W}, & s_u = -1. \end{cases} \quad (8)$$

The $+1$ in the second line is necessary for integer half-open intervals. Losing it creates genuine endpoint errors, especially for fixed residues.

4 The sharp interval-profile theorem

4.1 Residue counts are step functions

For each transported interval (a_j, ℓ_j, z_j) , divide its length by d :

$$\ell_j = q_j d + t_j, \quad 0 \leq t_j < d.$$

Let I_j be the cyclic interval of t_j consecutive residues starting at $a_j \bmod d$, empty when $t_j = 0$.

Lemma 4.1 (Exact residue sum). *The total weight of all points, over all base vertices in the block, whose root coordinates are congruent to r modulo d is*

$$F(r) = \sum_{j=1}^{K_i} (q_j + \mathbf{1}_{I_j}(r)) z_j, \quad (9)$$

where K_i is the number of records in that block. On the linearly cut residue interval $[0, d)$, F has at most $2K_i + 1$ constant cells. Its values and cell lengths can be computed without enumerating residues.

Proof. In a cyclic interval of length $q_j d + t_j$, each residue occurs q_j times and the first t_j residues occur once more. This remains true when the root interval wraps modulo W , since d divides W . Summing contributions gives (9) even for overlapping intervals and negative vector entries.

Each nonempty residual interval has two cyclic boundary events. After a cut at zero there are at most $2K_i$ internal event locations. Accumulate vector changes at these locations, sort them, and sweep the resulting cells. A wrapped residual interval is represented by its two ordinary pieces; endpoints zero and d need not be counted as new internal events. The operations depend on the number of records and their bit lengths, not on d or W as numeric quantities. $\square$

Without reflections, $F(r)$ is already the orbit weight. With reflections, define

$$G(r) = F(r) + F(c - r \bmod d). \quad (10)$$

A nonfixed residue pair has orbit weight $G(r)$. A fixed residue has orbit weight $F(r)$, not $G(r) = 2F(r)$.

4.2 Reflection quotient and a stronger support bound

A straightforward endpoint argument gives at most $4K_i + 1$ constant cells for G on $[0, d)$. That is a useful implementation bound but not the best support bound. Reflection symmetry halves the relevant changes.

Lemma 4.2 (Reflection quotient path). *For $d \geq 3$, the quotient of the discrete cycle on $\mathbb{Z}/d\mathbb{Z}$ by $r \mapsto c - r$ is a path, possibly with loops at its endpoints. Every edge on which an invariant function changes is paired with a distinct reflected edge. If an invariant function has at most $4K_i$ changing edges on the cycle, it takes at most $2K_i + 1$ distinct values on the quotient.*

Proof. For odd d , a translation of coordinates changes the reflection to $r \mapsto -r$. Writing $d = 2m + 1$, its vertex orbits are

$$\{0\}, \{\pm 1\}, \dots, \{\pm m\},$$

with consecutive adjacency and a possible loop at the last orbit. For even $d = 2m$, a translation makes c either zero or one. When $c = 0$, the sequence is

$$\{0\}, \{\pm 1\}, \dots, \{\pm(m-1)\}, \{m\}.$$

When $c = 1$, it is

$$\{0, 1\}, \{-1, 2\}, \dots, \{1 - m, m\}.$$

In both cases the nonloop quotient is a path.

A cycle edge fixed setwise by the reflection has its endpoints exchanged. An invariant function is equal at those endpoints, so it cannot change on that edge. Changing edges therefore have two-element reflection orbits. There are at most $2K_i$ corresponding changing edges along the quotient path. Deleting those edges partitions the path into at most $2K_i + 1$ constant runs, and hence bounds the number of distinct values. $\square$

Theorem 4.3 (Sharp-size weighted profiles). *A connected base block with K_i interval records has the following bounds on the support of its orbit-weight histogram:*

$$|\text{supp } H_i| \leq \begin{cases} 2K_i + 1, & \text{no reflection,} \\ 2K_i + 1 + f_i \leq 2K_i + 3, & \text{with reflection,} \end{cases} \quad (11)$$

where $f_i \leq 2$ is its number of fixed residues. For b base blocks and K total records,

$$|\text{supp } H| \leq 2K + b + \sum_i f_i \leq 2K + 3b, \quad (12)$$

using $f_i = 0$ in blocks without reflections. The histogram, including all binary multiplicities, is computable in polynomial bit time in the explicit structural parameters and $\log W$. No positivity assumption on the weights is needed.

Proof. The positive case follows from Lemma 4.1. In the signed case, each indicator $\mathbf{1}_{I_j}$ changes on at most two cycle edges. Its reflected copy does likewise. Therefore G changes on at most $4K_i$ edges. Cancellation among signed vectors can only remove changes. The function G is reflection-invariant, so Lemma 4.2 bounds its support by $2K_i + 1$. For $d = 1, 2$, the bound follows directly: there are at most two residues, and when $K_i = 0$ the function is identically zero.

Removing fixed residues cannot increase the support of the generic pair histogram. Replacing their doubled G values by their correct F values adds at most f_i values. This proves (11). Summing block supports proves (12); overlaps of weight values across blocks can only reduce the global support.

For computation, build F by its endpoint sweep. To build G , double the constant vector $\sum_j q_j z_j$ and add both I_j and its reflected interval. The latter has initial residue

$$c - a_j - t_j + 1 \pmod{d} \quad (13)$$

and length t_j . Thus the linearly represented G still needs only $O(K_i)$ cells. Accumulate each cell length by its vector value. For each fixed residue, subtract one from the multiplicity of its G value. All remaining multiplicities are even because reflection acts freely on the remaining residues and preserves G . Divide them by two and add one entry at each fixed residue's F value. These are operations on $O(K_i)$ integers and vectors, not loops over their magnitudes. $\square$

Remark 4.4 (Support versus the stored cell decomposition). The implementation is not required to store the quotient path explicitly. It uses the simpler $O(K_i)$ cyclic endpoint sweep, whose G representation can have up to $4K_i + 1$ linear cells. The sharper $2K_i + 3$ theorem bounds the *number of distinct histogram keys*. A larger internal cell constant does not invalidate that output bound.

4.3 A family attaining the bound

Theorem 4.5 (Sharpness with nonnegative scalar weights). *For every $K \geq 1$, there is a one-vertex system with K scalar interval records, all of weight $+1$, whose histogram has exactly $2K + 3$ distinct entries.*

Proof. Set

$$d = 4K + 4, \quad W = 5d,$$

and use the full-fibre maps $x \mapsto x + d$ and $x \mapsto -x$. Their translation divisor is d , and their two fixed residues are 0 and $d/2 = 2K + 2$. For $j = 1, \dots, K$, place a weight $+1$ on the ordinary, nonwrapping interval

$$[d - j, 4d + K + j + 1). \quad (14)$$

Its length is $3d + K + 2j + 1$, and its residual interval modulo d is the inclusive cyclic set $\{-j, -j + 1, \dots, K + j\}$. All endpoints in (14) lie in $[0, W]$.

At residue zero, every interval contributes 4, giving the fixed-orbit weight $4K$. At residue $2K + 2$, every interval contributes 3, giving weight $3K$. On the nonfixed quotient vertices $r = 1, \dots, 2K + 1$, interval j contributes to $G(r)$ the value

$$6 + \begin{cases} 2, & 1 \leq r \leq j, \\ 1, & j < r \leq K + j, \\ 0, & K + j < r \leq 2K + 1. \end{cases}$$

As r runs from 1 to $2K + 1$, precisely one unit is lost at each step: first at $j + 1$ for $j = 1, \dots, K$, then at $K + j + 1$ for $j = 1, \dots, K$. The generic weights are exactly

$$8K, 8K - 1, \dots, 6K.$$

They are pairwise distinct and disjoint from $3K, 4K$. Thus the support consists of $2K + 1$ generic values and two fixed values, proving the claim. $\square$

Example 4.6 (Nine profiles from three records). Taking $K = 3$ gives $d = 16, W = 80$, intervals $[15, 69), [14, 70), [13, 71)$, and histogram support

$$\{9, 12, 18, 19, 20, 21, 22, 23, 24\}.$$

Each multiplicity is one. The file `examples/sharp-nine-profiles.json` and its independently checked certificate realize this construction. The test suite checks the whole sharp family through $K = 30$ against explicit expansion.

4.4 Representatives without component enumeration

Proposition 4.7 (Exact representative selection). *Given a requested weight vector, and optionally t forbidden bulk-orbit keys, one can find a representative in an allowed orbit of that weight or prove that none exists, using a number of elementary operations polynomial in $K_i + t + D + B$. The method does not iterate over W , d , or the bulk orbit count.*

Proof. In the positive case, inspect constant cells of F with the requested value. Inside each candidate cell choose its first integer not in the forbidden set. A failed candidate advances only past an explicitly forbidden integer. Distinct cells are disjoint, so the total number of such advances is at most t .

In the reflection case, first test the at most two fixed residues using F . For generic components, inspect the constant cells of G , but exclude the fixed residues and both residues belonging to every forbidden orbit. At most $2t + 2$ integers are excluded. The same cell argument finds a residue without traversing a long interval; its canonical key is given by (7). Return that residue in the root fibre, where the gauge is the identity. $\square$

The result is stronger than a bare histogram for operational purposes: a caller can request one component satisfying a predicate on its weight, receive an actual named point, and retain that point in subsequent gluing records. It is weaker than a classification of the component's full attaching map, which is deliberately not inferred from the weight vector.

5 Sparse pointwise attachment replay

Let $\mathcal{O}$ be the set of bulk orbits and let $T \subseteq \mathcal{O}$ contain those touched by at least one endpoint of the h supplied attachments. Build a finite multigraph J on T with one edge for each attachment, using the exact bulk keys of its endpoints. Keep repeated edges, loops, and their payloads. There are at most $2h$ vertices.

Theorem 5.1 (Sparse quotient, histogram correction, and support). *The untouched bulk orbits remain final components. The other final components correspond exactly to the connected components A of J . For such a component, define*

$$W_A = \sum_{O \in V(A)} w(O) + \sum_{a \in E(A)} p_a. \quad (15)$$

Writing δ_z for the one-entry histogram of a single component of weight z , the final histogram and component count are

$$H' = H - \sum_{O \in T} \delta_{w(O)} + \sum_{A \in \pi_0(J)} \delta_{W_A}, \quad (16)$$

$$C' = C - |T| + |\pi_0(J)|. \quad (17)$$

Moreover,

$$|\text{supp } H'| \leq 2K + 3b + h. \quad (18)$$

All these data, connectivity queries, and representatives are obtainable with polynomial bit work in the stated explicit parameters, independently of the number of bulk orbits.

Proof. Quotient the lifted point set by the bulk equivalence relation first. Each new identification becomes precisely an edge of J . Taking transitive closure after that quotient is exactly graph connectivity on J . Untouched vertices of the bulk quotient acquire no new relations. This proves the component correspondence.

Every original point weight occurs in exactly one bulk orbit, and every attachment payload occurs in exactly one component of J . Thus additivity gives (15). Removing the old touched components and inserting the new clusters proves (16) and (17). Removal of histogram entries cannot increase support. Every connected component of J contains at least one edge, including the possibility of a loop, because T was defined by touched endpoints. Hence $|\pi_0(J)| \leq h$ when $h > 0$, and it is zero when $h = 0$. Theorem 4.3 now gives (18).

Exact keys come from the prepared gauges and residue formulas. Their weights come from F and its reflected counterpart. A disjoint-set forest processes the at most $2h$ keys. To find a representative for a requested final weight, first inspect the finite set of touched clusters. A stored key in a matching cluster is a valid root-fibre representative. If no touched cluster matches, use Proposition 4.7 on the bulk index, forbidding the touched bulk keys. Therefore no enumeration of untouched components is required. $\square$

5.1 The online invariant

When an attachment arrives, insert any previously untouched endpoint keys into a disjoint-set forest. Each forest root stores its current total vector weight. Remove the histogram entries of the endpoint roots; if they differ, join them and add their weights; then add the new edge payload and insert the resulting histogram entry. When both endpoints already belong to one cluster, remove and reinsert only that cluster's entry, but *still add the payload once*.

This last rule is easy to miss. A repeated attachment may not change connectivity, yet it may change Euler characteristic or another edge-additive statistic. Conversely, a repeated zero-payload attachment is a true no-op for both statistics. The implementation retains the supplied record sequence, so replay does not silently deduplicate geometric operations.

Corollary 5.2 (Cheap continuation predicates). *Given any polynomial-time predicate on component-weight vectors, the existence of a final component satisfying it can be decided by testing at most $2K + 3b + h$ values. If one exists, a representative can be returned. A predicate on these vectors is not automatically a predicate on full geometric continuation types.*

Proof. Evaluate the predicate on the support of (16), then apply the representative construction. The final qualification follows because the input weight vector need not encode a full attaching map or ambient embedding. $\square$

5.2 Why coordinates cannot be replaced by profile names

Example 5.3 (One self-attachment versus one joining attachment). Take eight separate bulk components, each of scalar weight 1. A payload- -1 attachment whose endpoints lie in the same component leaves seven components of weight 1 and one of weight 0. An attachment with the same payload whose endpoints lie in two distinct components leaves seven components of weight 1.

The endpoint weights, the number of attachment records, and the scalar payload are identical in these cases. Exact endpoint keys distinguish them. If the weights are certified Euler characteristics of surfaces and the attachments are certified boundary-arc gluings, the first operation does not create a disc, while the second can join two discs into one disc. The abstract example alone does not choose between an annulus and a Möbius band in the first case.

This gives a minimal obstruction to gluing by histograms alone. The histogram is an efficient query index; it is never the sole persistent geometric state. In particular, the linear support bound does not contradict the repository's example of exponentially many ordered two-point markings on one cyclic cover [6].

6 Insertion-only epochs and logarithmically many rebuilds

Fix the base graph, its spanning forest, all weight records, and all pointwise attachment endpoints. During an epoch, allow insertions of additional full-fibre signed-affine maps within existing base blocks. A map between two existing vertices of a block can be conjugated by their fixed gauges to a root-fibre map. The delivered dynamic API accepts the root form directly. It does not add new base vertices or change which base blocks are connected.

For each block keep the state (d, c) , where $c = \perp$ denotes the absence of a reflection and otherwise $0 \leq c < d$. A new root translation $r \mapsto r + t$ performs

$$d' = \gcd(d, t), \quad c' = \begin{cases} \perp, & c = \perp, \\ c \bmod d', & c \neq \perp. \end{cases}$$

A new reflection $r \mapsto -r + t$ performs

$$(d', c') = \begin{cases} (d, t \bmod d), & c = \perp, \\ (\gcd(d, t - c), t \bmod d'), & c \neq \perp. \end{cases}$$

In the second line $t \equiv c \pmod{d'}$, so storing $c \bmod d'$ instead is equivalent.

Theorem 6.1 (Effective-event bound). *Let block i begin with translation divisor d_i^0 and reflection state c_i^0 . During any insertion-only epoch, the number R of changes to the stored affine subgroup states satisfies*

$$R \leq \sum_{i=1}^b \left(\lfloor \log_2 d_i^0 \rfloor + \mathbf{1}_{\{c_i^0 = \perp\}} \right) \leq b(1 + \lfloor \log_2 W \rfloor). \quad (19)$$

Redundant maps require no profile reconstruction. Every effective event can be handled by rebuilding the affected block's interval profile and rebasing the sparse overlay from its original endpoint records. The total number of these rebuild triggers is independent of the total number q of inserted maps, although processing the q input records still has an unavoidable linear contribution.

Proof. The update formulas follow from Lemma 3.2: a new translation adds one increment, a first reflection introduces one reflection coset, and a later reflection adds the difference of its shift from the existing reflection.

Every changed divisor is a proper positive divisor of its previous value. Thus it is at most half the previous value. There can be at most $\lfloor \log_2 d_i^0 \rfloor$ such changes in block i . A first reflection can occur at most once. After a reflection is present, c cannot change without a proper divisor drop: a redundant reflection has $t - c \equiv 0 \pmod{d}$, and a redundant translation leaves d unchanged. This proves the first bound; $d_i^0 \leq W$ proves the second.

If (d, c) is unchanged, the generated full-fibre action is unchanged, hence so are all orbit keys and all weighted profiles. On a changed state, a new endpoint sweep computes the new block profile. Old pointwise attachments are then reinterpreted by their saved, exact fibre endpoints and Theorem 5.1 applies. No step requires finding new geometric attachments or enumerating the old bulk components. $\square$

The bound concerns the stored signed-affine subgroup state. In the small fibres $W = 1, 2$, formally different signed maps can induce the same permutation. Similarly, adding a reflection when $d = 1$ changes the stored state without changing connectivity. These harmless extra rebuilds do not invalidate the upper bound; we do not claim that every counted event strictly reduces the number of components.

Example 6.2 (An extremal stored-state chain). With $W = 2^m$ and no initial maps, successively insert translations by $2^{m-1}, 2^{m-2}, \dots, 1$, followed by a reflection. There are $m + 1$ stored-state changes. Inserting arbitrarily many redundant copies between these steps changes the input size but not the number of rebuild events. The test suite checks this pattern with $m = 12$.

6.1 A precise amortized accounting statement

Let A_i be the cost of constructing the profile for block i from its cached transported weight intervals. Let U bound replay of all current h pointwise attachments, and let $G(B)$ bound one gcd and the accompanying binary arithmetic. Apart from initial gauge construction and the cost of explicitly requested output, the epoch cost is bounded by

$$O(qG(B)) + \sum_{i=1}^b (R_i + 1)A_i + (R + 1)U, \quad (20)$$

where R_i counts effective events at block i . This is a conservative bound: it permits rebasing the whole sparse overlay after an event in a single block. A more selective implementation could avoid replaying attachments disjoint from the affected block.

The source list still retains all q maps. Storing a redundant map is an append operation; creating a complete source snapshot, serializing it, or producing its final certificate costs at least the size of that snapshot. Formula (20) does not pretend that every full-history certificate can be produced after every update for free.

6.2 Stale keys, cancellation, and epochs that really end

The producer increments an equivalence-version counter on an effective state change. A sparse overlay refuses queries against a newer bulk version until it has rebased from the original endpoints. Keeping a disjoint-set forest on the old numerical residue labels would be unsound: two formerly distinct bulk orbits may now coincide, and their old weights cannot simply be added as though the underlying point sets remained disjoint.

Profile reconstruction is prepared separately before a dynamic state is committed. Sparse rebasing is likewise constructed separately, so cooperative cancellation cannot expose a partly replayed overlay as a complete new state. A cancelled rebase leaves a stale overlay that must still reject queries. Individual integer operations and the independent certificate pass are not hard-interruptible.

Insertion-only monotonicity is essential. If deletion is allowed, alternate between adding and deleting the translation $x \mapsto x + 1$ in a large trivial fibre system. The component partition alternates forever between one component and W components. A fixed $\log W$ bound on all rebuilds is then false. Cutting a hierarchy can reset the base graph or the coordinate system, and is therefore an epoch boundary unless a separate continuation theorem proves otherwise.

7 A checked front end for ordinary interval pairings

The full-fibre promise should be tested, not guessed from a suggestive diagram. Suppose a caller supplies equal-width fibre blocks and partitions its claimed full-map rules into groups of nonwrapping interval isometries. A member of a group has data

$$(u, z, \varepsilon, [a, b), y),$$

meaning that source integer $x \in [a, b)$ maps to $y + \varepsilon(x - a)$ in the target fibre. Require both endpoint images to lie in $[0, W)$.

Theorem 7.1 (Exact full-fibre guard). *For a supplied group, the following checks certify that its union is precisely one full-fibre map $x \mapsto \varepsilon x + t \pmod{W}$:*

1. *all records have the same source, target, and sign;*
2. *all residues $y - \varepsilon a \pmod{W}$ agree;*
3. *the source intervals, after sorting, partition $[0, W)$ without gaps or overlap;*
4. *each individual interval map is nonwrapping and remains in the target fibre.*

Every full-fibre signed-affine map admits such a group with at most two intervals. The guard uses polynomial bit time in the number of supplied records and their bit lengths.

Proof. Conditions 1 and 2 give one common modular formula. Conditions 3 and 4 establish that its restrictions are exactly the supplied maps on every source integer, once each. Therefore their union is that full modular map, which is a bijection because its sign is ± 1 .

For a translation with normalized shift t , split at $W - t$ when $t \neq 0$; no split is needed when $t = 0$. For a reflection $x \mapsto -x + t$, split at $t + 1$ unless $t = W - 1$. On each piece the ordinary integer formula does not wrap. Sorting and endpoint arithmetic give the claimed cost. $\square$

This checks a *supplied* partition into blocks and map groups. It neither discovers a common fibre coordinate in an arbitrary normal-surface presentation nor shows that every relevant presentation has one. Any original pairing not covered by the full-map groups must be separately accounted for. A residual interval of length L is not automatically one sparse defect: without further compression it contains L pointwise identifications, and that quantity must be charged to h .

The API raises `UnsupportedModel` when the promise fails. An unsupported input, a resource limit, an invalid certificate, or a timeout is never converted into a knot verdict. A host recognizer should retain its existing exact fallback.

8 Bit complexity and certificate soundness

8.1 No numeric-size iteration

Put $S = v + e + K + h + 2$. With B as in Section 2, all normalized residues, divisor values, counts, and accumulated vector coordinates use $O(B)$ bits, up to a constant factor. For example, an orbit weight has absolute coordinate at most

$$KW \max_j \|z_j\|_\infty + h \max_a \|p_a\|_\infty,$$

whose bit length is bounded by the input bit lengths plus $O(\log S)$. Counts are at most vW before attachments and never increase under attachment identifications.

Gauge construction uses $O(v + e)$ modular operations. The residual subgroup needs $O(e)$ gcd updates. Preparing the interval profiles uses $O(KD)$ vector-coordinate arithmetic together with $O(K \log(K + 2))$ endpoint comparisons, with constant-factor duplication for reflections. A prepared point-weight query uses at most two searches among $O(K_i)$ endpoints in its block and $O(D)$ coordinate arithmetic. Sparse replay touches at most $2h$ bulk keys and uses a disjoint-set structure of that size.

To state a conservative deterministic bit bound without relying on Python hash-table average behavior, one may use comparison dictionaries or allow worst-case quadratic dictionary work. Ordinary schoolbook arithmetic and the Euclidean algorithm then give

$$T_{\text{static+attachments}} = O(S^2(D + 1)B^3) \quad (21)$$

as a safe, deliberately nonoptimal bound. The internal arrays, source records, cells, and histogram require $O(S(D + 1)B)$ bits apart from language-object overhead. The bound explicitly excludes any iteration proportional to the numeric values W or d , or to the number of lifted components.

The reference implementation uses standard-library dictionaries and unbounded integers. Its measured behavior can be better than the conservative bound, but no deterministic near-linear bit bound is inferred from the use of a hash table. Likewise, no claim is made that this specialized implementation beats the general Agol–Hass–Thurston algorithm on their broader input class.

8.2 Certificate contents

A certificate contains a schema identifier and a source digest, rooted-forest arrays, gauge signs and shifts, one residual subgroup record for each base block, gcd witnesses, each block’s histogram, and the final attachment histogram and count. The source consists of the complete normalized model plus the original pointwise attachments, including multiplicities and payloads.

At a gcd step on nonnegative integers a, b , the producer supplies g, x, y satisfying

$$g > 0, \quad g \mid a, \quad g \mid b, \quad xa + yb = g. \quad (22)$$

These equations force $g = \gcd(a, b)$: every common divisor divides the Bézout combination, while g itself divides both inputs. The checker therefore does not need to run the producer’s gcd computation.

Theorem 8.1 (Arithmetic certificate soundness). *If the delivered independent checker accepts a source and a certificate, the claimed final histogram and component count equal those of the exact signed-affine fibre system and attachment records in that source. A valid certificate of polynomial bit length exists for every valid input in the stated explicit-base regime. Verification has polynomial bit cost in the source and certificate lengths and explicit structural parameters.*

Proof. The checker validates the input schema, signs, intervals, dimensions, and endpoint ranges. It checks that every nonroot parent edge joins distinct vertices, satisfies its claimed gauge equation, and reaches a declared root without a parent cycle. It also checks that every original edge has both endpoints in one declared tree block. Thus the forest spans precisely the base connected components and (3) is valid for the supplied gauges.

For every original edge, the checker independently derives the residual map. It verifies the full gcd chain by (22), which fixes d , and checks the reflection residue. Lemma 3.2 then fixes the bulk orbit relation.

The checker does not call the producer’s endpoint-sweep code. It constructs candidate endpoints directly from the transported intervals and, on each cell, evaluates (9) by counting the contribution of every input interval. With reflections, it evaluates both terms of (10), removes fixed residues,

halves paired multiplicities, and reinstates the correct fixed weights. Its computed block histogram is therefore exact. This deliberately simple independent computation costs $O(K_i^2 D)$ arithmetic rather than sharing the faster sweep.

Finally, the checker constructs the graph of touched bulk keys, traverses its connected components independently of the producer's disjoint-set forest, and sums each payload once. Theorem 5.1 proves its final histogram. Acceptance requires exact equality to the claimed histograms and count.

All arrays and witnesses have polynomial length. Extended Euclid provides suitable coefficients of polynomial bit length; the profile support is linear by Theorem 4.3; and sparse data have size polynomial in h . Polynomial verification follows from the explicitly bounded arithmetic and finite graph operations. An adversarial oversized certificate is charged by its actual length, not by an assumed honest size. $\square$

8.3 Trusted boundary and limits of replay

The producer and checker share strict schema and hexadecimal transport utilities. They do not share the profile algorithm, gcd discovery, or sparse connectivity implementation. The checker uses Python's integer arithmetic and its own graph traversal; it is not a Lean formalization. Exhaustive tests and a second algorithm reduce implementation risk but do not replace the mathematical proof.

The source SHA-256 digest is an accidental-misbinding guard, not the sole basis for correctness. The checker derives and checks the arithmetic against the actual supplied source. It verifies an orbit histogram, not a geometric extraction from a knot exterior. A certificate accepted here cannot by itself certify a disc's embedding, slope, normal-coordinate validity, or relation to an original knot diagram.

Integer fields reject booleans, floats, and decimal strings. They accept literal integers and signed hexadecimal strings; output uses hexadecimal transport so that huge exact integers do not require changes to Python's process-wide decimal conversion limit. Malformed inputs are errors, not answers to an unrelated topological decision problem.

9 A conditional surface interpretation and disc completion

The quotient construction can support a useful geometric rule, but only with an explicit interface. Suppose the bulk orbits index actual compact connected surfaces with nonempty boundary. Suppose their exact Euler characteristics have been supplied as additive weights, and every new attachment represents a gluing of two disjoint boundary arcs with the relevant manifold and disjointness conditions checked. Give each such attachment scalar payload -1 .

Proposition 9.1 (Surface defect on the attachment quotient). *For a connected component A of the attachment multigraph, let S_A be the resulting compact connected surface, assumed to have nonempty boundary. Then*

$$\chi(S_A) = \sum_{O \in V(A)} \chi(S_O) - |E(A)|, \quad (23)$$

$$1 - \chi(S_A) = \sum_{O \in V(A)} (1 - \chi(S_O)) + \beta_1(A). \quad (24)$$

Consequently S_A is a disc exactly when every S_O is a disc and A is a tree.

Proof. A gluing along two disjoint boundary arcs decreases Euler characteristic by one. Applying the operation for every edge gives (23); loops and repeated edges each contribute separately. Since A is connected, $\beta_1(A) = |E(A)| - |V(A)| + 1$, which gives (24).

For a compact connected surface with nonempty boundary, $1 - \chi$ is nonnegative and vanishes exactly for the disc. This follows from the standard classification formulas $\chi = 2 - 2g - b$ in the orientable case and $\chi = 2 - k - b$ in the nonorientable case. The graph quantity $\beta_1(A)$ is also nonnegative, vanishing exactly for a tree. Thus the right side of (24) vanishes precisely in the claimed situation. $\square$

The defect identity is a standard Euler-characteristic accounting principle, not claimed here as a new disc-completion theorem. The new implementation consequence is that its finite attachment quotient can be evaluated over an exponentially large supplied affine bulk system without enumerating its untouched components. A disc-weight query can return an exact component anchor for further geometric verification.

This proposition has intentionally strong hypotheses. Polygon edges sharing vertices require careful cell-counting corrections; arbitrary point identifications do not satisfy the arc-gluing formula automatically. Gluing entire boundary circles has a different Euler contribution. A closed spherical component has positive Euler characteristic but is not a disc. A disc abstractly present in a surface system need not be a properly embedded spanning or compression disc in the original knot exterior. All these distinctions must remain visible in the host interface.

10 Validation and measured kernel behavior

10.1 Checks actually executed

The package was executed on CPython 3.13.5. The final unit suite has 34 tests and passes. Several tests contain exhaustive or randomized inner loops, so the unit-test count alone understates the number of comparisons. The suite includes all 816 ordered pairs of signed-affine generators on $W = 1, \dots, 8$; 800 random multivertex systems; 1,800 random full-map updates across 60 epochs; all 930 signed-affine maps through $W = 30$ for the interval-splitting guard; and the sharp scalar construction for $K = 1, \dots, 30$.

The finite oracle explicitly builds the lifted point graph and computes components by graph traversal. It independently assigns interval weights point by point. This deliberately expensive oracle is restricted to small fibres and is not part of production. It checks component weights, connectivity, representatives, reflection fixed-point corrections, repeated attachments, loops, and signed payloads.

A separate recorded audit, with seed 261009703, checks 2,000 additional random models. It performs 19,818 pointwise attachment insertions, 179,684 point-weight comparisons, 100,000 connectivity comparisons, 7,760 representative checks, and 2,000 independent certificate replays. Every check passes. Raw counts, source hashes, and elapsed time are in `results/audit.json`.

Tests also mutate source weights, omit attachments, alter forest signs and gcd witnesses, change divisors and histogram multiplicities, introduce Boolean aliases, and duplicate histogram entries. These changes are rejected. A replay test disables the producer constructor to demonstrate that checking does not invoke it. Large-integer tests use $W = 2^{24000}$ and exact hexadecimal certificate round trips. Cancellation tests check both atomic full-map updates and a rebase interrupted after part of its work.

These finite checks establish what was executed, not universal empirical coverage. The correctness theorems supply the general argument. No maintained ProveIt recognizer suite, no native AHT implementation, and no knot corpus benchmark was run for this delivery. An optional documented comparison script against the maintained `CoverIndex` public component-count API is included but explicitly marked unexecuted.

10.2 Paired experiment design

The benchmark uses a fixed seed, one complete untimed warmup per arm, and five shuffled measured rounds. Each experiment has a reference strategy, an optimized strategy, and an identical optimized control. Every arm must return exactly the same result. Fixtures are generated outside the timer; construction and requested histogram queries are timed; output equality checks and JSON serialization are outside the timer. Static, sparse, and epoch timings exclude certificate generation and replay. A separate experiment includes those operations.

The reference arms are deliberately named rather than passed off as prior maintained versions. Static experiments compare with the independent explicit-sheet oracle. The sparse experiment compares rebuilding the supplied model and replaying every prefix against one prepared index with incremental insertions. The epoch experiment compares reconstruction after every new full map against reconstruction only on effective subgroup changes. Neither repeated-rebuild reference is the current maintained ProveIt recognizer.

Experiment	Reference ms	Optimized ms	Paired ratio	A/A ratio
$W = 256$	1.353	0.241	5.67	1.05
$W = 1024$	6.048	0.320	19.50	1.20
$W = 4096$	29.307	0.453	61.66	0.98
$W = 16384$	166.322	0.548	319.50	0.68
96 sparse prefixes	146.742	9.208	17.59	0.88
200 full-map updates	202.847	21.202	9.65	0.72

Table 1: Five-round kernel medians and median paired time ratios. Static rows use four base vertices, 32 weight records, and dimension three. Sparse and epoch rows use $W = 2^{1024}$ and 64 records of dimension four. “Paired ratio” is reference/optimized; “A/A” is identical-control/optimized.

The static rows illustrate the growth of explicit-sheet work, not an improvement over another compressed orbit algorithm. The sparse-prefix run has 96 attachments and 96 requested prefix histograms. The epoch run has 200 inserted full maps and 24 fixed sparse attachments; exactly nine insertions change the subgroup state. The structural reconstruction count drops from 200 to nine update-triggered profile rebuilds, in addition to initial preparation.

The observed paired ratios are approximately 17.6 for the sparse-prefix experiment and 9.65 for the epoch experiment. These are finite measurements of the stated strategies. Identical-arm controls range from roughly 0.68 to 1.20, with especially substantial variation in the short static and epoch measurements. Thus the experiment does not support precise machine-independent speed factors or small constant-factor comparisons. All raw samples and execution orders are retained; the noisy controls are not suppressed. The mathematical avoidance of repeated reconstruction, rather than these exact ratios, is the robust conclusion.

10.3 Large-bit complete certificate runs

The separate large-bit experiment uses four vertices, 32 interval records, dimension three, and one pointwise attachment. It measures index construction, attachment insertion, certificate production, and independent replay. It excludes JSON serialization, whose output size is nevertheless recorded.

The $W = 2^{24000}$ case completes its producer and checker in a median of about 51 milliseconds in this run. The certificate itself occupies 823,491 bytes, reflecting the real cost of retaining large integer coordinates. This is a successful arithmetic-kernel test, not a test on a knot with 2^{24000}

Sheet count	Complete median ms	Final support	Certificate bytes
2^{1024}	6.293	67	42,904
2^{4096}	12.790	67	148,893
2^{24000}	51.229	66	823,491

Table 2: Three complete producer-and-checker runs per sheet count. Byte counts use compact hexadecimal JSON for the certificate alone, excluding the separately supplied source.

crossings and not a general topology performance claim.

11 Repository integration and the quasi-polynomial target

11.1 Proposed integration boundary

The delivery is a standalone research package with no import-time dependency on ProveIt. Its entry points are `BulkIndex`, `SparseOverlay`, the pairing guard, certificate production, and independent replay. Proposed integration should first keep it as an opt-in supplied-model component kernel. It should not replace the maintained cover classifier, which computes additional surface topology and peripheral data outside the present scope.

A host-side adapter needs to produce five things: an explicit base graph and common fibre size, exact signed-affine full maps with a supplied interval partition, interval credits for every queried additive statistic, precise endpoint records for every sparse attachment, and provenance linking all these data to the host’s original geometric objects. Our guard validates only the combinatorial full-map portion. The arithmetic certificate checks the supplied model and its attachments; it does not verify that provenance.

For an initial comparison, `integration/check_native_cover.py` constructs compatible bouquet systems and compares their component counts with the maintained `fastunknot.surface_cover.CoverIndex`. That script is not a full topology equivalence test and was not run here. Native integration should subsequently compare complete source-bound operations, including extraction, preparation, replay, and fallback costs, on genuine geometric instances. No performance claim for the maintained recognizer should be made from the synthetic tables alone.

11.2 What the kernel contributes to a complexity proof

Corollary 11.1 (Conditional end-to-end accounting). *Suppose an exact unknot-recognition procedure on inputs of size n has a verified schedule of at most $2^{O((\log n)^2)}$ states. Suppose each state supplies a correct fibre-system instance of the present type with $v, e, K, D, h, B \leq n^{O(1)}$, and all other state construction, verification, transition, and fallback costs together are at most $2^{O((\log n)^2)}$. Replacing the corresponding component-query tasks by the kernels proved here preserves a quasi-polynomial end-to-end bound, even when W itself is exponential in a polynomial of n .*

Proof. Equation (21) is polynomial in the bounded per-state parameters. The product of a polynomial and $2^{O((\log n)^2)}$ is still $2^{O((\log n)^2)}$. Since B charges $\log W$ rather than W , the stated large multiplicities do not change that calculation. All other required work is covered by the explicit hypotheses. $\square$

This corollary is accounting, not a new bound on the number of unknot-recognition states. Its value is the removal of two hidden assumptions: that weighted profiles require component

enumeration, and that every inserted full map requires a fresh full orbit computation. The fixed-base epoch theorem further bounds reconstruction within a verified monotone portion of a schedule. Neither theorem bounds how often a hierarchy cuts, repairs, changes its fibre model, or restarts an epoch.

11.3 Three nonimplications that must survive integration

First, a linear number of weight vectors does not mean a linear number of attachment configurations. The exact coordinates retained by the sparse overlay may have many distinct values. Second, a polynomial algorithm for checking a supplied model is not a polynomial algorithm for discovering a suitable model. Third, an abstract disc query is not a complete unknot certificate. The ambient embedding and peripheral boundary conditions remain necessary.

The repository already documents a related distinction for its ordered marked-cover signatures: compact unmarked classification does not bound the number of marked states [5, 6]. The sharp profile theorem is consistent with that warning. It controls additive orbit statistics, not the entire geometric continuation relation.

12 Further research questions and proposed next steps

12.1 A checked geometric block producer

The highest-value next step is a producer that identifies equal-width affine blocks in an actual normal-surface or parallelity-bundle presentation, emits the interval guard records, and proves that every original pairing is accounted for. A useful first theorem would cover a clearly defined family of layered handle structures, including a bound on the number of residual pointwise exceptions. A producer without a checked reconstruction of the original pairings would not close the gap.

12.2 Compressed residual intervals rather than point defects

Can the h -point attachment theorem be extended to p additional partial affine intervals with a bound polynomial in p and their binary lengths? A direct expansion makes h depend on interval length and loses the intended compression. A successful extension must control the quotient of these partial intervals by the bulk action and identify precisely when that quotient still has a short endpoint representation. General weighted AHT processing is a natural fallback and a necessary comparison baseline.

12.3 Variable-width fibres and arithmetic interfaces

Real decompositions may have different fibre sizes at different vertices. An extension could allow compatible sublattices and affine maps between specified arithmetic progressions. The key question is whether an explicitly checked coordinate-lattice structure still permits a linear or polynomial bound on weighted profiles. Arbitrary changes of modulus should not be assumed to retain the reflection-quotient argument.

12.4 Selective rebasing and a dependency graph

The current safe strategy rebases all sparse attachments on every effective event. One can track which attachment clusters depend on each base block and rebuild only those affected. Prove an amortized bound charging each affected attachment to a strictly decreasing divisor or a union in a

dependency structure. The complication is that an existing sparse cluster can touch several blocks, so a local bulk change can propagate through old attachments.

12.5 Deletions, cuts, and epoch potentials

The simple insertion-only argument cannot handle hierarchy cuts. A meaningful replacement would be a global potential that charges both subgroup growth inside an epoch and the cost of resetting its coordinate model. An important intermediate target is a bounded-depth hierarchy in which each reset decreases a certified geometric complexity. Merely summing a logarithmic bound over an unbounded number of epochs gives no useful recognition guarantee.

12.6 Peripheral data and orientation characters

The present weights may include certified additive quantities, but neither cyclic boundary order nor orientation propagation is automatically additive interval data. Combine the maintained cover classifier's boundary and orientation interface with this kernel while preserving exact attachment coordinates. A theorem should state when the richer profile support remains bounded and when pointwise queries, rather than a complete profile list, are the appropriate output.

12.7 Beyond signed translations

For maps $x \mapsto ax + t \pmod{W}$ with arbitrary invertible a , the quotient of the residue cycle is no longer a path, and an interval can have a much more complicated image. Determine a natural subclass admitting an analogue of the sharp support theorem, or construct explicit support-growth obstructions. Any claim of a uniform extension must address this loss of one-dimensional reflection geometry.

12.8 Certificates for incremental histories

Current certificates describe a final source-bound state. A compact incremental proof log could certify a stream of insertions, no-op decisions, and occasional rebases without resubmitting an entire history at each query. Its verifier must still account for every original attachment payload and every source change. A promising target is a checkable gcd-update chain plus persistent endpoint records, with an explicit total proof-size bound.

12.9 Exact bridge to disc-completion representative bases

A disc-completion dynamic program may already keep a small representative family of boundary states. Investigate whether the weighted-orbit quotient can supply its local component and defect queries while retaining the exact boundary transport needed for continuation. The open issue is not the Euler identity alone, but compatibility between the program's equivalence relation on states and the fibre model's exact point labels.

12.10 Native benchmarks and falsifiable performance criteria

Before enabling this kernel in the maintained recognizer, measure complete extraction-plus-query workloads against both the existing component path and a general compressed orbit algorithm. Include instances that fail the full-fibre guard, systems with many effective events, many sparse attachments, and cases where setup dominates. Record paired controls and negative outcomes.

A concrete success criterion is a reduction in end-to-end verified component-query time on a preregistered corpus without changing any recognizer verdict or weakening resource accounting.

12.11 The remaining global theorem

The ultimate target is a verified producer and schedule satisfying Corollary 11.1 for every unknot-recognition input. The sharp support and epoch results eliminate particular multiplicity-dependent costs under their contracts. Proving that all relevant geometric states admit those contracts, or a comparably efficient generalization, remains a substantive research task. The next result should address one of those quantified hypotheses rather than restating the conditional product bound.

13 Conclusion

A signed-affine bulk system with interval weights is more structured than a general compressed orbit problem. Its reflection quotient yields the exact sharp support bound $2K + 3$, even for scalar positive weights in the extremal family. A finite graph on touched orbit keys then handles arbitrary explicit pointwise attachments with the support bound $2K + 3b + h$, without losing the coordinates needed to distinguish self-attachments from joins. Fixed-base monotone insertion further limits expensive reconstruction to logarithmically many subgroup changes.

These are proved, implemented improvements to a specific component-query interface that can be tested and integrated independently. The implementation and its checker establish no general quasi-polynomial unknot bound on their own. Their contribution is to make one part of such a proof more explicit: small output, exact provenance at the pointwise attachment level, bounded rebuild triggers, bit-costed operations, and independently replayable arithmetic certificates.

A Source audit and comparison boundary

The repository head observed during review was

b23481370a2cc00332d1f69bc881ccf107d25c7c.

The files were fetched from the live default branch during that review; their content identities are recorded below rather than assuming every fetch was an atomic checkout. The full fetched theory and interface text were read. The top-level README was used for project scope, not as a source of a current maintained test total.

Fetches item	Content identity or review limit
README.md (project root)	Git blob 7d80dc0740b232d01c113e26c156e6ac2370d92b.
fast/cover_research/README.md under the same project	Git blob 1c73d13ec6ce3edf3db993e1fbc6ade817a8ffd9.
synthesis/surface_covers.tex	Git blob 151dc30bf343e99403c16632b6eced3bebeeee90c.
docs/incoming/README.md	Initial 100 lines inspected; Git blob a776280b5408eda46be78afb687dfb4731fb496c.
Incoming archive listing	Names, sizes, and Git blob identities only; archive contents not audited.

The metadata-visible archives include ProveIt_Disc_Completion_Bases_2026-10-09.zip, ProveIt_Disk_Completion_Kernels_2026-10-09.zip, and ProveIt_Signed_Continuation_

Bases_20261009.zip. Their existence is not treated as evidence for any mathematical claim in this manuscript. No unseen archive content is paraphrased as though it had been examined. A later intake review should compare this package’s exact sharp-support theorem and insertion/replay interface with those manuscripts.

Primary external sources checked were the full weighted-orbit theorem page of Agol–Hass–Thurston, the hierarchy preprint’s Section 9, and the covering-space classification in Hatcher. Their source PDFs and figures are not redistributed with this delivery. The accompanying SOURCE_AUDIT.md records the same boundary and source addresses.

B Reproduction and artifact map

From the package root, the following commands need only the Python standard library except for PDF compilation:

```
PYTHONPATH=src python -m unittest discover -s tests -v
python scripts/audit.py
python scripts/benchmark.py
python scripts/make_examples.py
python scripts/make_tables.py
PYTHONPATH=src python -m affine_orbits \
  examples/sharp-nine-profiles.json --output result.json
PYTHONPATH=src python -m affine_orbits \
  examples/sharp-nine-profiles.json --verify result.json
cd article && latexmk -pdf -interaction=nonstopmode article.tex
```

src/affine_orbits/core.py implements gauges, interval sweeps, exact keys, sparse replay, and dynamic epochs. guard.py checks full-map decompositions. certificate.py produces final-state certificates. checker.py independently reconstructs arithmetic and sparse connectivity. tests/oracle.py is an intentionally explicit finite oracle and is not a production algorithm.

results/unit_tests.txt, audit.json, and benchmarks.json record completed executions. The two generated table files derive from the recorded benchmark JSON rather than manually entered timings. The optional native comparison script and integration notes state that native checks were not executed. The checksum manifest covers all delivered files except itself. No generated Python cache directories, font files, third-party PDFs, or LaTeX temporary files are part of the archive.

References

- [1] I. Agol, J. Hass, and W. P. Thurston, *The computational complexity of knot genus and spanning area*, arXiv:math/0205057v2, 2004. See Theorem 12, Theorem 16, and Corollary 17. <https://arxiv.org/abs/math/0205057>.
- [2] A. Hatcher, *Algebraic Topology*, Cambridge University Press, 2002. Section 1.3, especially Theorem 1.38. Author-hosted text: <https://pi.math.cornell.edu/~hatcher/AT/AT.pdf>.
- [3] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026. See Section 9. <https://arxiv.org/abs/2607.23350>.

- [4] V. Reshetnikov, ProveIt repository, *Topology/UnknotRecognition/README.md*, reviewed 9 October 2026. <https://github.com/VladimirReshetnikov/ProveIt/tree/main/Topology/UnknotRecognition>.
- [5] ProveIt project, *Binary surface covers and ordered marked points*, *Topology/UnknotRecognition/fast/cover_research/README.md*, reviewed 9 October 2026; content identity in Appendix A. https://github.com/VladimirReshetnikov/ProveIt/blob/main/Topology/UnknotRecognition/fast/cover_research/README.md.
- [6] ProveIt project, *Compressed surface covers and ordered attachment points*, *Topology/UnknotRecognition/synthesis/surface_covers.tex*, reviewed 9 October 2026; content identity in Appendix A. https://github.com/VladimirReshetnikov/ProveIt/blob/main/Topology/UnknotRecognition/synthesis/surface_covers.tex.